

EXECUTIVE_PRESENTATION_AND_DEPL OYMENT_GUIDE

NetworkPeer Executive Presentation and Deployment Guide

This is the operating guide for the NetworkPeer (GigGrid) field-work marketplace. It covers the backend through Phase 12, the live browser workflow, and the shortest safe path to a production-like demonstration.

1. Executive Summary: Backend Architecture and Logic

NetworkPeer is a secure, event-driven field-work marketplace. It is designed around a simple business rule: a worker may receive private job details and complete work only after the client funds escrow and PostgreSQL accepts the exact lifecycle transition.

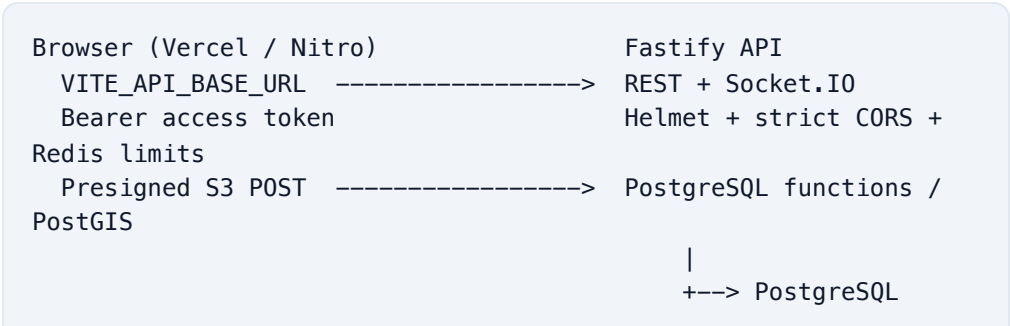
What the platform demonstrates

Capability	Why it matters in the presentation
PostGIS spatial discovery	Workers search nearby funded jobs using geography-aware radius queries. Public results intentionally hide the client identity, exact address, exact coordinates, and exact distance until acceptance.
Atomic job acceptance	PostgreSQL's <code>accept_job</code> function locks and validates the job in one transaction. Concurrent workers race

Capability	Why it matters in the presentation
	safely: one receives 200 , one receives 409 , and exactly one worker is assigned.
Pure lifecycle rules	TypeScript state-machine functions and database constraints agree on legal job progressions. Application code cannot skip from an arbitrary status to completion.
Native cryptographic JWTs	Access and refresh tokens use pinned HS256 signing, constant-time signature comparison, issuer/audience checks, and explicit expiry validation.
Redis refresh rotation	Refresh tokens are single-use sessions. Replay of an old token revokes its full token family, which limits the blast radius of stolen refresh credentials.
Durable synchronization	PostgreSQL records ordered recipient-specific sync events before Socket.IO broadcasts them. The browser recovers missed notifications using a cursor API.
S3 Media Engine	Evidence is first reserved, then uploaded using a short-lived presigned POST. The backend validates MIME type, exact length, SHA-256, ETag, version ID, and lifecycle tag before accepting it.
BullMQ with durable outboxes	PostgreSQL media and push outboxes are authoritative. Redis/BullMQ accelerates delivery and retry, but a periodic database sweep recovers missed jobs after queue loss or restart.
Escrow and immutable ledger	Funding, escrow release, platform fee, worker payout, and payout reversals are represented as balanced double-entry journal

Capability	Why it matters in the presentation
	postings. Gateway webhooks settle the business state.
Least-privilege database roles	Normal API, admin, media verification, and financial workflows use distinct database roles and narrowly granted <code>SECURITY DEFINER</code> functions.
Phase 10 observability	Fastify and background workers emit redacted Pino JSON logs. Sentry captures unhandled HTTP failures, promise rejections, and process exceptions when a DSN is configured.
Phase 10 protection	Helmet adds security headers. Strict origin CORS permits only configured browser origins. <code>@fastify/rate-limit</code> uses Redis for global request limits while OTP flows retain stricter phone-specific limits.
Phase 11 E2E coverage	The E2E suite creates and funds a job, processes a signed payment webhook, races two workers through the API, and proves that only one atomic acceptance succeeds.
Phase 12 operations	A multi-stage Docker image, production Compose topology, migration/provisioning target, worker process, and GitHub Actions verification workflow are ready in the repository.

Architecture at a glance



durable outboxes and ledger	+--> Redis / BullMQ
workers	+--> S3 versioned
evidence	+--> Stripe Connect
webhooks	+--> FCM push
delivery	+--> Sentry +
structured Pino logs	

Data-authority rule

PostgreSQL is authoritative for jobs, funding, evidence acceptance, ledger postings, push state, and queue state. Redis is not a financial or workflow source of truth. If Redis is restarted, the API and worker recover outstanding work from PostgreSQL.

2. Rapid Deployment Plan: Getting It Live Tonight

Use Stripe **test mode** for tomorrow's demonstration. Do not use live payment credentials or a real customer card during a presentation.

2.1 Preflight checklist

1. Have an AWS S3 bucket with versioning enabled, default encryption enabled, and all public-access blocks enabled.
2. Have a Stripe test-mode account, test secret key, test webhook signing secret, and Stripe CLI installed locally.
3. Have a Twilio account if demonstrating real SMS. For a local rehearsal only, use `SMS_PROVIDER=console` and `OTP_ECHO_IN_RESPONSE=true`; never use either setting in production.
4. Have a managed PostgreSQL service where PostGIS can be enabled and a managed Redis endpoint that supports TLS (`rediss://`).
5. Select a backend hostname and frontend hostname before setting CORS. The API accepts exact origins only, not wildcard origins.
6. Use two browser profiles for the demo: a normal profile for the client and a private/incognito profile for the worker. Browser sessions are intentionally tab/profile scoped.

2.2 Generate real secrets once

Run these commands locally. Store each output in Railway/Render/Vercel secrets or the platform's encrypted environment-variable UI. The values are URL-safe hexadecimal strings and work safely inside PostgreSQL and Redis

connection URLs.

```
openssl rand -hex 48 # JWT_SECRET
openssl rand -hex 48 # JWT_REFRESH_SECRET
openssl rand -hex 32 # POSTGRES_PASSWORD for Docker Compose
only
openssl rand -hex 32 # REDIS_PASSWORD for Docker Compose only
openssl rand -hex 32 # NETWORKPEER_APP_DB_PASSWORD
openssl rand -hex 32 # NETWORKPEER_ADMIN_DB_PASSWORD
openssl rand -hex 32 # NETWORKPEER_MEDIA_DB_PASSWORD
openssl rand -hex 32 # NETWORKPEER_FINANCIAL_DB_PASSWORD
openssl rand -hex 48 # PAYMENT_WEBHOOK_SECRET
```

Never commit the resulting values. The repository ignores `.env` and `.env.local`.

2.3 Fastest managed deployment: Railway or Render

The following process works with Railway, Render, Fly.io, or any equivalent PaaS. Use the provider's public TLS PostgreSQL and Redis connection strings for hosted deployment. Do **not** set

`ALLOW_INSECURE_INTERNAL_TRANSPORT=true` on a PaaS.

1. Create a managed PostgreSQL database named `networkpeer`.
2. Confirm the database owner can run `CREATE EXTENSION postgis` or enable PostGIS through the provider's extension control.
3. Create a managed Redis service and use its TLS URL beginning with `rediss://`.
4. Create an API service from this repository with root directory `NetworkPeer-main`.
5. Use the Dockerfile builder, or set build command `npm ci && npm run build` and start command `node dist/index.js`.
6. Create a second service from the same backend source for workers. Use start command `node dist/background-worker.js`.
7. Give the worker the same secrets as the API. Set `BACKGROUND_QUEUES_ENABLED=false` on the API service and `BACKGROUND_QUEUES_ENABLED=true` on the worker service.
8. Before either service receives traffic, run migrations and provision the database roles from a trusted terminal using the migration-owner TLS connection.

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
npm ci
export DATABASE_URL="$DATABASE_MIGRATION_URL"
```

```
npm run migrate
psql "$DATABASE_MIGRATION_URL" -v ON_ERROR_STOP=1 -f
scripts/provision-app-role.sql
```

`DATABASE_MIGRATION_URL` is the provider's migration-owner connection string. It is not used by the running API. The provisioning script reads the four `NETWORKPEER_*_DB_PASSWORD` variables directly from the environment and creates passwords without exposing them in a psql command line.

- Construct four TLS application connection strings using the provider's host/database and the four role names created by the provisioning script:

Runtime variable	Database role
<code>DATABASE_URL</code>	<code>networkpeer_app</code>
<code>DATABASE_ADMIN_URL</code>	<code>networkpeer_admin_api</code>
<code>DATABASE_MEDIA_VERIFIER_URL</code>	<code>networkpeer_media_verifier</code>
<code>DATABASE_FINANCIAL_URL</code>	<code>networkpeer_financial_api</code>

Every hosted connection string must include `sslmode=require` or stronger. The role names are fixed; only the host, database, and secret values come from your provider.

- Deploy the API and worker. Verify the API's public `GET /api/v1/live` endpoint returns `200` before deploying the browser.

2.4 Deploy the frontend on Vercel

- Create a Vercel project using repository root directory `NetworkPeer-platform-main`.
- Set Node.js 24 in the Vercel project settings. The verified Nitro Vercel build emits a `nodejs24.x` server function.
- Set build command `npm run build`.
- Set `NITRO_PRESET=vercel`. The frontend's Lovable/TanStack configuration recognizes Nitro's Vercel target at build time.
- Set `VITE_API_BASE_URL` to the exact public API URL followed by `/api/v1`.
- Set `VITE_API_PREFIX=/api/v1`.
- Deploy the frontend and copy the exact HTTPS Vercel origin.
- Set backend `CORS_ORIGINS` to that exact origin, without a trailing slash, then redeploy the API and worker.

Vite exposes only `VITE_*` variables to browser code. Never place Stripe secret keys, database URLs, Twilio secrets, Sentry server secrets, or AWS credentials in Vercel variables.

2.5 Required production environment variables

Set these values in both the API and worker unless a row says otherwise.

Variable	Required value or rule
<code>NODE_ENV</code>	<code>production</code>
<code>PORT</code>	The PaaS listener port; use the platform-provided value when available.
<code>API_PREFIX</code>	<code>/api/v1</code>
<code>DATABASE_URL</code>	TLS URL for <code>networkpeer_app</code> .
<code>DATABASE_ADMIN_URL</code>	TLS URL for <code>networkpeer_admin_api</code> .
<code>DATABASE_MEDIA_VERIFIER_URL</code>	TLS URL for <code>networkpeer_media_verifier</code> .
<code>DATABASE_FINANCIAL_URL</code>	TLS URL for <code>networkpeer_financial_api</code> .
<code>DATABASE_MIGRATION_URL</code>	Migration-owner TLS URL; set only on the one-shot migration/provisioning job or trusted terminal.
<code>NETWORKPEER_APP_DB_PASSWORD</code>	Role password supplied only while provisioning <code>networkpeer_app</code> .
<code>NETWORKPEER_ADMIN_DB_PASSWORD</code>	Role password supplied only while provisioning <code>networkpeer_admin_api</code> .
<code>NETWORKPEER_MEDIA_DB_PASSWORD</code>	Role password supplied only while provisioning <code>networkpeer_media_verifier</code> .
<code>NETWORKPEER_FINANCIAL_DB_PASSWORD</code>	Role password supplied only while provisioning

Variable	Required value or rule
	<code>networkpeer_financial_api</code> .
<code>DATABASE_POOL_MIN</code>	<code>0</code> to <code>2</code> , based on service size.
<code>DATABASE_POOL_MAX</code>	Maximum simultaneous PostgreSQL clients per process. Start with <code>10</code> .
<code>REDIS_URL</code>	Managed TLS URL beginning with <code>rediss://</code> .
<code>JWT_SECRET</code>	First 48-byte hexadecimal secret generated above.
<code>JWT_REFRESH_SECRET</code>	Different 48-byte hexadecimal secret generated above.
<code>JWT_ACCESS_TTL</code>	<code>15m</code>
<code>JWT_REFRESH_TTL</code>	<code>7d</code>
<code>JWT_ISSUER</code>	<code>networkpeer-api</code>
<code>JWT_AUDIENCE</code>	<code>networkpeer-mobile</code>
<code>OTP_ECHO_IN_RESPONSE</code>	<code>false</code>
<code>SMS_PROVIDER</code>	<code>twilio</code>
<code>TWILIO_ACCOUNT_SID</code>	Twilio account SID.
<code>TWILIO_AUTH_TOKEN</code>	Twilio auth token.
<code>TWILIO_FROM_NUMBER</code>	Twilio E.164 sender number.
<code>AWS_REGION</code>	Region containing the evidence bucket.
<code>AWS_S3_BUCKET</code>	Private, versioned, encrypted S3 bucket name.
<code>AWS_ACCESS_KEY_ID</code>	Leave blank only when the runtime has an IAM task/instance role.
<code>AWS_SECRET_ACCESS_KEY</code>	Leave blank only when the runtime has an IAM task/instance role.

Variable	Required value or rule
<code>AWS_SESSION_TOKEN</code>	Optional temporary IAM session token.
<code>PAYMENT_GATEWAY</code>	<code>stripe</code>
<code>PAYMENT_WEBHOOK_SECRET</code>	Long independent webhook secret for non-Stripe/stub compatibility.
<code>STRIPE_SECRET_KEY</code>	Stripe test key for tomorrow; live key only after go-live approval.
<code>STRIPE_WEBHOOK_SECRET</code>	Signing secret of the configured Stripe webhook endpoint.
<code>STRIPE_CONNECT_CLIENT_ID</code>	Stripe Connect client ID.
<code>PAYMENT_DISPATCH_ENABLED</code>	<code>true</code>
<code>BACKGROUND_QUEUES_ENABLED</code>	<code>false</code> on API-only replicas; <code>true</code> on worker replicas.
<code>BACKGROUND_QUEUE_POLL_INTERVAL_MS</code>	<code>5000</code>
<code>BACKGROUND_MEDIA_CONCURRENCY</code>	<code>2</code> initially.
<code>BACKGROUND_PUSH_CONCURRENCY</code>	<code>4</code> initially.
<code>PUSH_NOTIFICATIONS_ENABLED</code>	<code>false</code> unless all Firebase variables are configured.
<code>FIREBASE_PROJECT_ID</code>	Required only when push is enabled.
<code>FIREBASE_CLIENT_EMAIL</code>	Required only when push is enabled.
<code>FIREBASE_PRIVATE_KEY</code>	Required only when push is enabled. Preserve newline escapes.
<code>CORS_ORIGINS</code>	Exact comma-separated HTTPS browser origins.
<code>TRUST_PROXY_CIDRS</code>	Only the actual reverse-proxy IP/CIDR ranges. Leave empty for a direct listener.

Variable	Required value or rule
<code>REALTIME_ENABLED</code>	<code>true</code>
<code>SENTRY_DSN</code>	Sentry Node project DSN, or empty to disable Sentry intentionally.
<code>SENTRY_ENVIRONMENT</code>	<code>production</code>
<code>SENTRY_RELEASE</code>	Deployed Git SHA or release identifier.
<code>SENTRY_TRACES_SAMPLE_RATE</code>	Start with <code>0.1</code> .
<code>RATE_LIMIT_WINDOW_MS</code>	<code>60000</code>
<code>RATE_LIMIT_MAX_REQUESTS</code>	Start with <code>100</code> ; set lower values at edge/WAF for unauthenticated paths.
<code>LOG_LEVEL</code>	<code>info</code>
<code>LOG_PRETTY</code>	<code>false</code>
<code>ALLOW_INSECURE_INTERNAL_TRANSPORT</code>	<code>false</code> on all hosted deployments.
<code>POSTGRES_PASSWORD</code>	Docker Compose database-owner password only; never set on managed runtime services.
<code>REDIS_PASSWORD</code>	Docker Compose Redis password only; never set when <code>REDIS_URL</code> comes from a managed provider.

2.6 Configure S3 browser upload CORS

The worker browser uploads directly to the presigned S3 POST URL. Add the exact frontend origin to the bucket CORS policy. Run this from a terminal where `AWS_S3_BUCKET` and `FRONTEND_ORIGIN` contain real values.

```
cat > /tmp/networkpeer-s3-cors.json <<EOF
{
  "CORSRules": [
    {
      "AllowedHeaders": ["*"],
      "AllowedMethods": ["POST"],
```

```

    "AllowedOrigins": ["$FRONTEND_ORIGIN"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 300
  }
]
}
EOF
aws s3api put-bucket-cors --bucket "$AWS_S3_BUCKET" --cors-
configuration file:///tmp/networkpeer-s3-cors.json

```

Keep bucket public access blocked. The browser never receives general S3 credentials; it receives a reservation-specific, time-limited POST policy only.

2.7 Docker deployment option

For a self-hosted production-like stack, use the supplied files in

`NetworkPeer-main`.

```

docker compose --env-file .env.prod -f docker-compose.prod.yml up
--build -d
docker compose --env-file .env.prod -f docker-compose.prod.yml ps
docker compose --env-file .env.prod -f docker-compose.prod.yml
logs -f api worker

```

`docker-compose.prod.yml` starts PostGIS, password-protected Redis, a one-shot migration/provisioning service, the Fastify API, and a separate background worker. It accepts only generated URL-safe secrets through `.env.prod`; it has no committed default passwords.

`ALLOW_INSECURE_INTERNAL_TRANSPORT=true` is limited by code to Compose's private `postgres` and `redis` hostnames and must never be copied to a PaaS deployment.

2.8 CI/CD behavior

`.github/workflows/deploy.yml` runs on pull requests, pushes to `main`, and manual dispatch. It starts disposable PostGIS/Redis services, installs dependencies, runs lint/typecheck/build/unit tests, migrates an isolated `networkpeer_e2e` database, runs the funded atomic-acceptance E2E test, and builds the production runtime Docker image.

3. Frontend and Backend Integration Guide

Browser configuration checklist

1. Set `VITE_API_BASE_URL` at frontend build time to the public API origin with `/api/v1` appended.

2. Set backend `CORS_ORIGINS` to the exact frontend origin. Do not add a wildcard and do not include a trailing slash.
3. Redeploy the backend after changing CORS, then rebuild/redeploy the frontend after changing a `VITE_*` value. Vite variables are compiled into the browser bundle.
4. Confirm `GET /api/v1/live` from the frontend origin succeeds and browser DevTools shows `Access-Control-Allow-Origin` matching the frontend origin.
5. Preserve the API prefix. The frontend client already maps paths such as `/client/jobs` to `<VITE_API_BASE_URL>/client/jobs`.
6. Use HTTPS for frontend, API, and Socket.IO. Production configuration rejects non-HTTPS CORS origins.
7. Configure the S3 CORS policy above before testing browser evidence uploads.

Authentication contract

The frontend's `src/lib/api.ts` reads the active session and sends this header automatically on authenticated calls:

```
Authorization: Bearer <access-token>
```

The browser handles a `401` by rotating the refresh token through `POST /api/v1/auth/refresh` once, then retries the original request. It removes its session only when a refresh is rejected. Tokens remain in browser session storage, not durable local storage.

For local rehearsal, `OTP_ECHO_IN_RESPONSE=true` causes the real development OTP to appear on the verification page. Production returns no OTP and sends the code through Twilio.

Live UI capabilities now wired to the API

Screen	Live behavior
Client create job	Creates a real <code>FUNDING</code> job with a GeoJSON point, budget, and checklist.
Client job detail	Starts funding, displays the Stripe test <code>PaymentIntent</code> reference, cancels while funding, and approves submitted work.

Screen	Live behavior
Worker nearby/job detail	Uses PostGIS discovery, accepts the job through the atomic database function, then links to the live task screen.
Worker live task	Advances ASSIGNED -> EN_ROUTE -> AT_LOCATION -> IN_PROGRESS , calculates SHA-256, reserves evidence, performs the S3 POST, confirms evidence, and submits work.
Client wallet	Requests server-calculated balances derived from immutable ledger postings.

API flow reference

```
POST /client/jobs
POST /client/jobs/:jobId/fund
Stripe test confirmation -> POST /webhooks/payments
GET /worker/jobs/nearby
POST /worker/jobs/:jobId/accept
POST /work/status
POST /work/upload-url
POST <presigned S3 URL>
POST /work/evidence
POST /work/submit
POST /client/jobs/:jobId/approve
GET /client/wallet or /worker/wallet
```

Realtime integration

The frontend derives the Socket.IO origin from VITE_API_BASE_URL . Socket.IO uses the API origin, while REST requests retain /api/v1 . Keep the load balancer’s WebSocket upgrade support enabled. If a Socket.IO event is missed, the browser uses the durable sync cursor endpoint rather than assuming realtime delivery is complete.

4. Live Demo Script: Step-by-Step Walkthrough

4.1 Demonstration preparation

Complete this before the meeting.

1. Deploy the API, worker, frontend, managed database, Redis, and S3 configuration.
2. Open the frontend in a normal browser profile and a private/incognito profile.
3. Confirm the API health endpoints:

```
curl -fsS "$API_ORIGIN/api/v1/live"  
curl -fsS "$API_ORIGIN/api/v1/health"
```

4. Provision an admin one time using the migration-owner connection:

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main  
psql "$DATABASE_MIGRATION_URL" -f scripts/provision-admin.sql
```

5. Sign up the worker in the private browser. Workers are intentionally not allowed to accept work until verified.
6. Verify the worker through the audited database function. Export real admin and worker phone numbers first, then run:

```
psql "$DATABASE_MIGRATION_URL" \  
--set=admin_phone="$ADMIN_PHONE" \  
--set=worker_phone="$WORKER_PHONE" <<'SQL '  
SELECT public.admin_set_worker_verification(  
  (SELECT id FROM public.users WHERE phone_number =  
    : 'admin_phone'),  
  (SELECT id FROM public.users WHERE phone_number =  
    : 'worker_phone'),  
  'VERIFIED',  
  TRUE,  
  'Verified for executive live demonstration'  
);  
SQL
```

7. Configure Stripe CLI forwarding to the public backend before the funding step:

```
stripe login  
stripe listen --forward-to "$API_ORIGIN/api/v1/webhooks/payments"
```

Copy the webhook signing secret shown by Stripe CLI into

`STRIPE_WEBHOOK_SECRET` , then redeploy the API and worker if it changed.

4.2 Opening statement

Say this before clicking anything:

NetworkPeer does not trust browser state for job assignment, money, or evidence. The browser is a workflow client; PostgreSQL, the ledger, and versioned evidence records remain the source of truth.

Step A: Log in as a client and create/fund a job

1. In the normal browser profile, open the frontend and choose **Client**.
2. Enter the client phone number and complete OTP verification.
3. Open **Create a job**.
4. Enter a title, a description of at least ten characters, a budget, and a location. Use the same nearby coordinates the worker will use.
5. Keep one required checklist item such as "Capture storefront evidence".
6. Click **Create job**.
7. Explain the displayed **FUNDING** status: the job is intentionally invisible to workers until escrow settles.
8. Open the created job and click **Fund escrow**.
9. Copy the displayed Stripe test PaymentIntent ID.
10. In a terminal authenticated to the same Stripe test account, confirm that exact payment intent:

```
stripe payment_intents confirm "$STRIPE_PAYMENT_INTENT_ID" --  
    payment-method pm_card_visa
```

11. Wait for Stripe CLI to forward `payment_intent.succeeded` to the API.
Refresh the client job page until the status becomes **POSTED**.

What to say:

The client created a job, but it was not published. A signed Stripe webhook settled the escrow hold in PostgreSQL, generated a balanced ledger journal, and only then made the job eligible for worker discovery.

Step B: Log in as a worker, search nearby, and accept atomically

1. In the private browser profile, choose **Worker** and complete OTP verification using the verified worker account.
2. Ensure the worker location is close to the client's job coordinates.
3. Open the worker dashboard and show the nearby job returned by the PostGIS search.

4. Open the job. Point out that it initially protects client identity and only presents privacy-safe discovery data.
5. Click **Accept job**.
6. The UI reveals the protected location and checklist only after acceptance.
7. Optional concurrency proof: open the same posted job in a second verified worker profile and click **Accept job** at the same time. One request succeeds and the other receives a conflict.

What to say:

Acceptance is not a browser race. Both requests reach a PostgreSQL function that locks the row and validates worker eligibility. The database chooses exactly one winner.

Step C: Upload evidence through S3 and update database metadata

1. On the accepted worker job, click **Continue live task**.
2. Click **Mark EN ROUTE**, then **Mark AT LOCATION**, then **Mark IN PROGRESS**.
3. Under the required evidence checklist, click **Capture or select evidence**.
4. On mobile, use the camera capture prompt. On desktop, select a small JPEG or PNG.
5. Watch the status change to **Evidence confirmed**.
6. In AWS S3, show the new `evidence/` object version if time allows. Do not make it public.
7. Click **Submit evidence for review**.

What to say:

Before S3 receives the file, the API reserves its allowed content type, exact byte size, and SHA-256. After upload, the API reads the immutable S3 version and validates the metadata again. The database creates a durable media-processing outbox row, and BullMQ performs background work without making Redis authoritative.

Step D: Approve the work and show wallet ledger updates

1. Return to the client browser profile.
2. Refresh the client job detail. Its status should be `SUBMITTED`.
3. Click **Approve and release payout**.

4. Open **Wallet** and click **Refresh balances**.
5. Show the real, server-calculated currency summary: available balance, escrow, and lifetime spend.
6. If time permits, show API/worker logs with the payment operation ID and a structured JSON record.

What to say:

Approval does not rewrite a balance. It creates immutable, balanced release, fee, and payout postings. The worker payout is dispatched independently and moves to final state only when the gateway webhook confirms it.

4.3 Live recovery commands

Use these commands if the demo environment needs a quick diagnosis.

```
# API liveness and dependency health
curl -i "$API_ORIGIN/api/v1/live"
curl -i "$API_ORIGIN/api/v1/health"

# Local Docker topology
docker compose --env-file .env.prod -f docker-compose.prod.yml ps
docker compose --env-file .env.prod -f docker-compose.prod.yml
  logs --tail=100 api worker

# Database migration state
psql "$DATABASE_MIGRATION_URL" -c "SELECT filename, applied_at
  FROM schema_migrations ORDER BY filename;"

# Queue transport only; durable state remains in PostgreSQL
redis-cli -u "$REDIS_URL" ping
```

4.4 Final close

The platform combines a clean marketplace experience with backend controls that remain correct under retries, concurrent workers, Redis restarts, payment webhooks, and versioned media uploads. The demo UI is not a mock layer: its critical actions are the same authenticated API calls and database functions used in production.

Verification Before Presentation

Run these in order on a non-production database before presenting.

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
npm run lint
npm run typecheck
npm run build
npm test
npm run migrate
npm run test:e2e
```

The E2E command requires `DATABASE_URL` to reference a database whose name contains `test` or `e2e`. This safety check prevents cleanup code from ever targeting a production database.

For frontend verification:

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-platform-
  main
npm run lint
npm run build
```